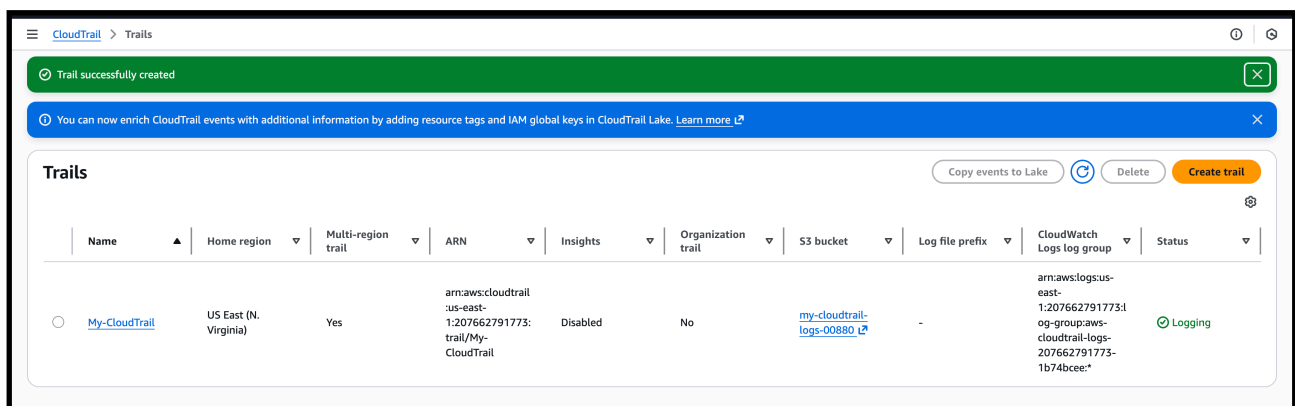


task on monitoring cloutrail cloud watch

1) Enable cloutrail monitoring and store the events in s3 and cloudwatch log events.

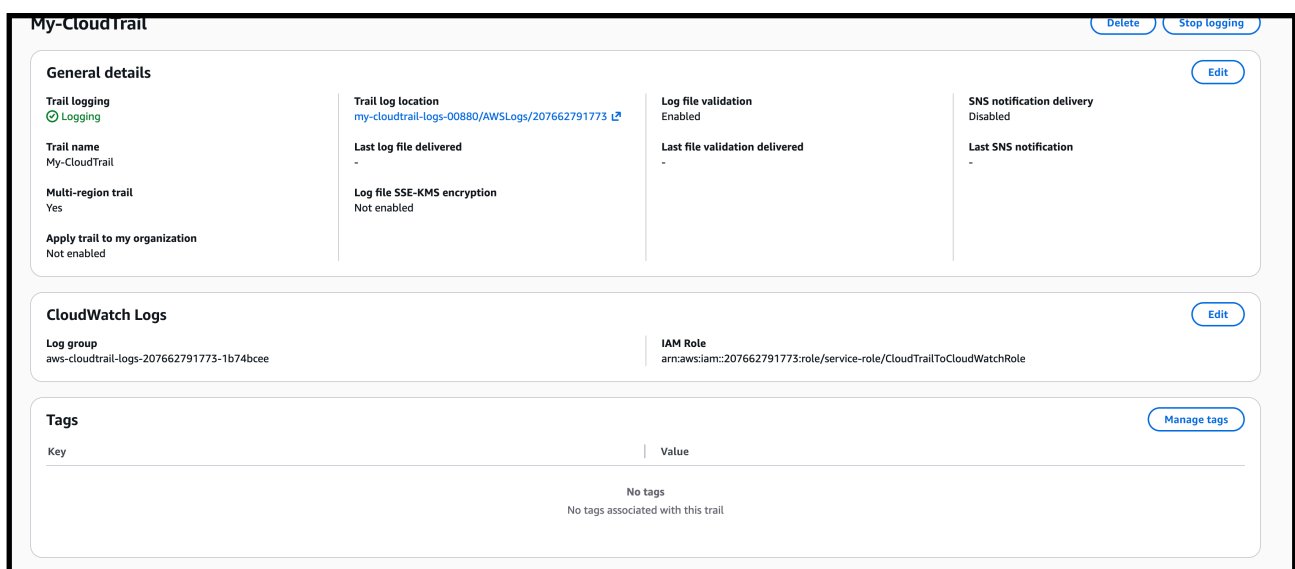
—> created a cloutrail monitoring and storing the events in s3 bucket



The screenshot shows the AWS CloudTrail Trails console. At the top, there is a green notification bar stating "Trail successfully created" and a blue information bar about enriching events with resource tags and IAM global keys in CloudTrail Lake. Below these, the "Trails" section displays a table with columns: Name, Home region, Multi-region trail, ARN, Insights, Organization trail, S3 bucket, Log file prefix, CloudWatch Logs log group, and Status. A single trail named "My-CloudTrail" is listed with the following details: Home region is US East (N. Virginia), Multi-region trail is Yes, ARN is arn:aws:cloudtrail:us-east-1:207662791773:trail/My-CloudTrail, Insights are Disabled, Organization trail is No, S3 bucket is my-cloudtrail-logs-00880, Log file prefix is -, CloudWatch Logs log group is arn:aws:logs:us-east-1:207662791773:log-group:aws-cloudtrail-logs-207662791773-1b74bcee, and Status is Logging.

Name	Home region	Multi-region trail	ARN	Insights	Organization trail	S3 bucket	Log file prefix	CloudWatch Logs log group	Status
My-CloudTrail	US East (N. Virginia)	Yes	arn:aws:cloudtrail:us-east-1:207662791773:trail/My-CloudTrail	Disabled	No	my-cloudtrail-logs-00880	-	arn:aws:logs:us-east-1:207662791773:log-group:aws-cloudtrail-logs-207662791773-1b74bcee	Logging

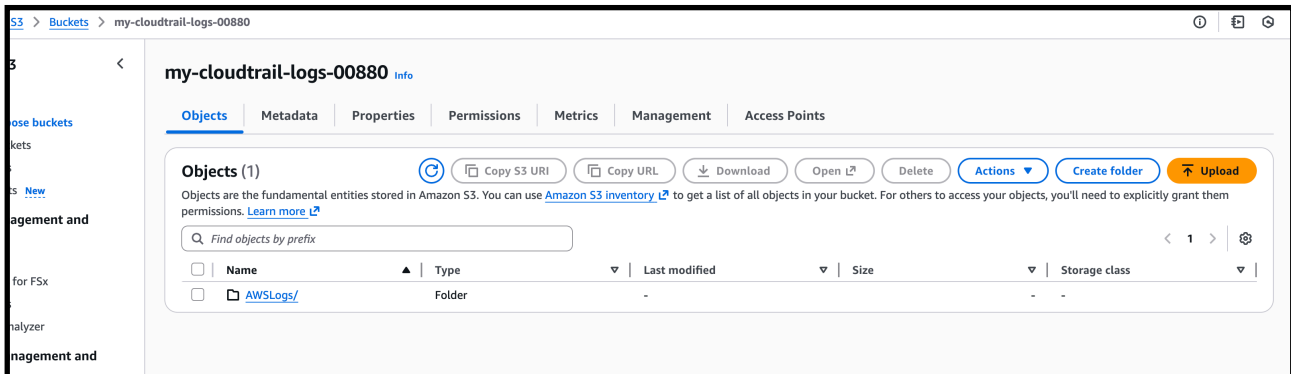
—> created cloud trail details with s3 bucket detail and cloud watch details



The screenshot shows the details for the "My-CloudTrail" trail. It is divided into three main sections: General details, CloudWatch Logs, and Tags. The General details section includes fields for Trail logging (Logging), Trail name (My-CloudTrail), Multi-region trail (Yes), Apply trail to my organization (Not enabled), Trail log location (my-cloudtrail-logs-00880/AWSLogs/207662791773), Last log file delivered (-), Log file SSE-KMS encryption (Not enabled), Log file validation (Enabled), Last file validation delivered (-), and SNS notification delivery (Disabled). The CloudWatch Logs section shows the Log group (aws-cloudtrail-logs-207662791773-1b74bcee) and the IAM Role (arn:aws:iam::207662791773:role/service-role/CloudTrailToCloudWatchRole). The Tags section shows no tags associated with this trail.

Key	Value
No tags	
No tags associated with this trail	

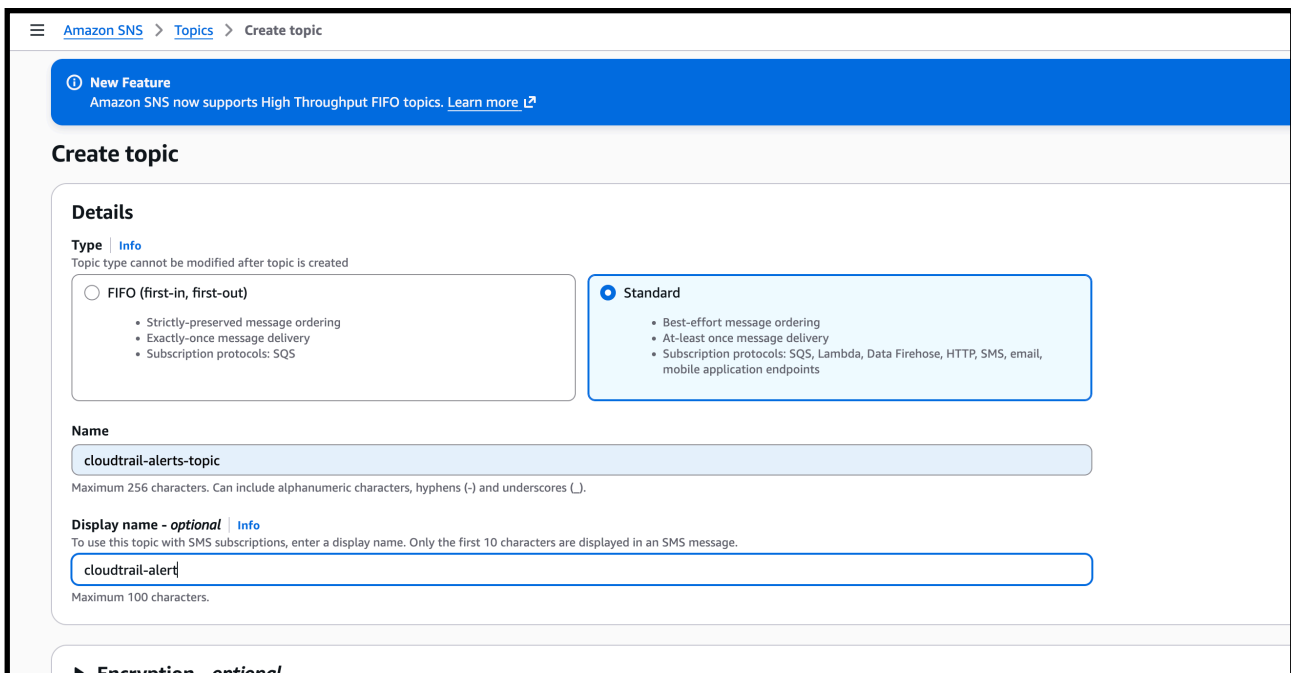
—> the cloudtrail logs are getting stored in s3 bucket



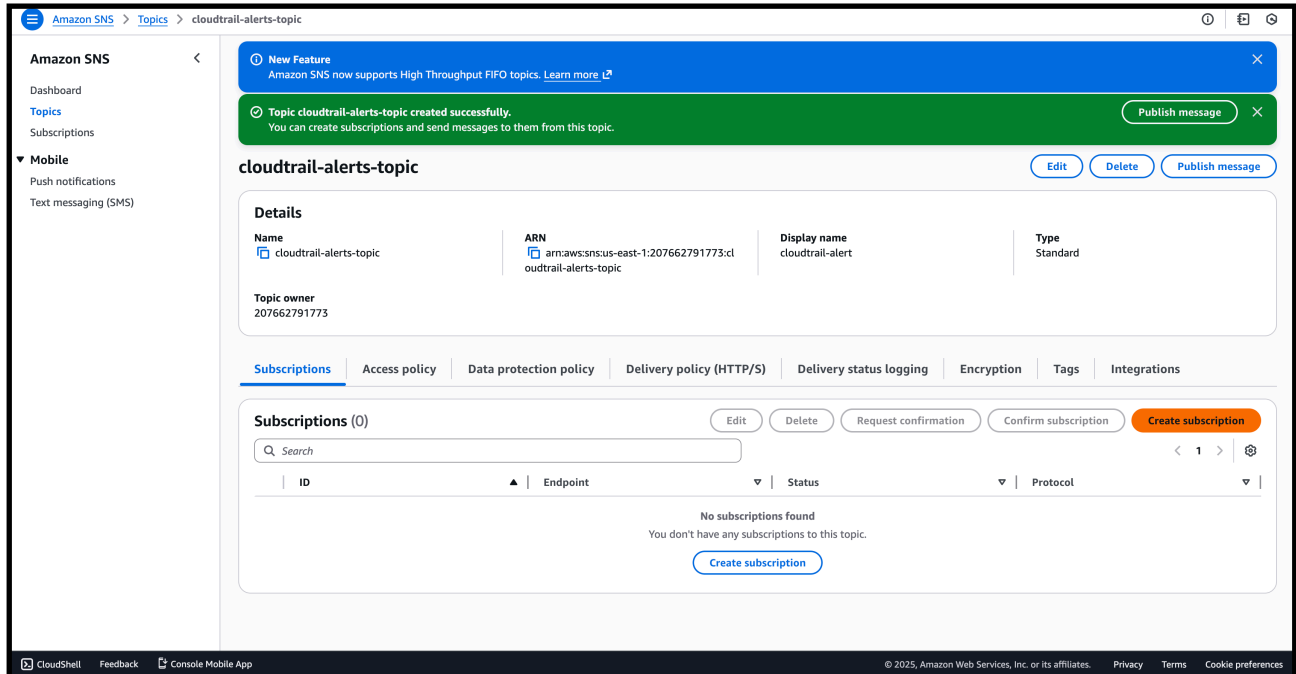
2) Enable SNS for cloudtrial to send alert on email.

—> create a sns topic

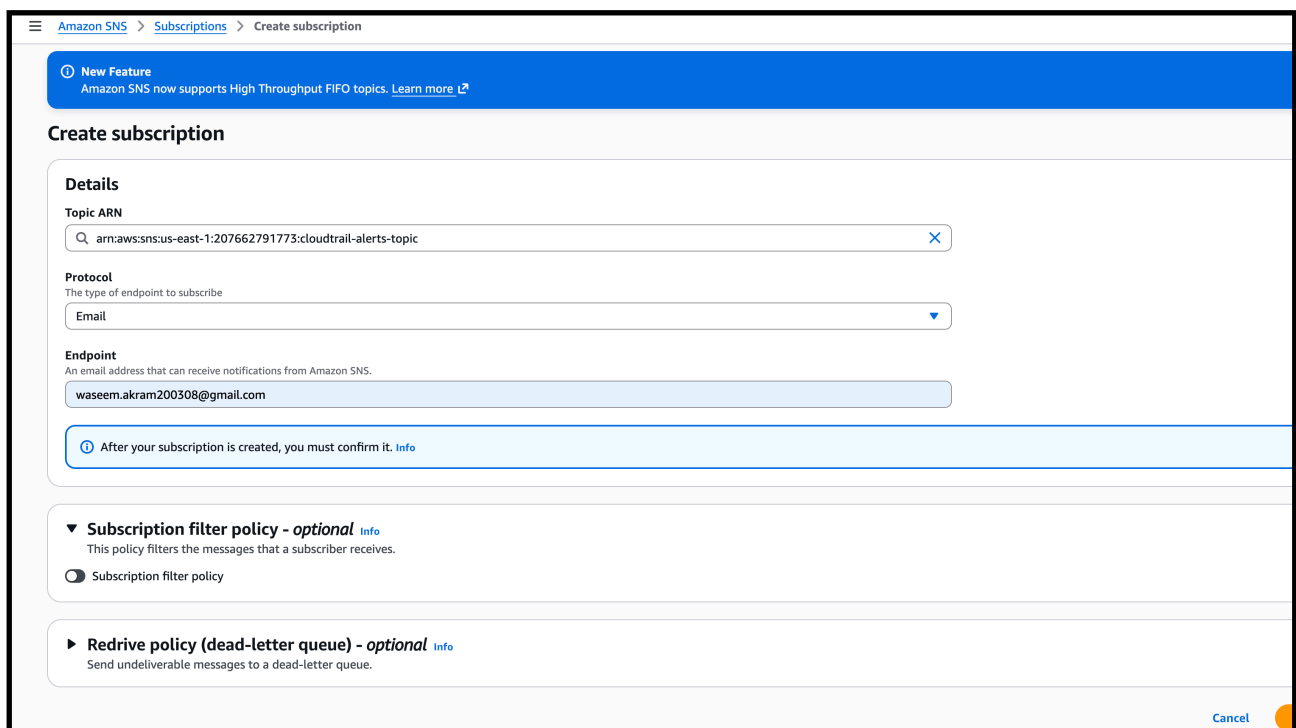
—> give it a name cloudtrail-alerts-log



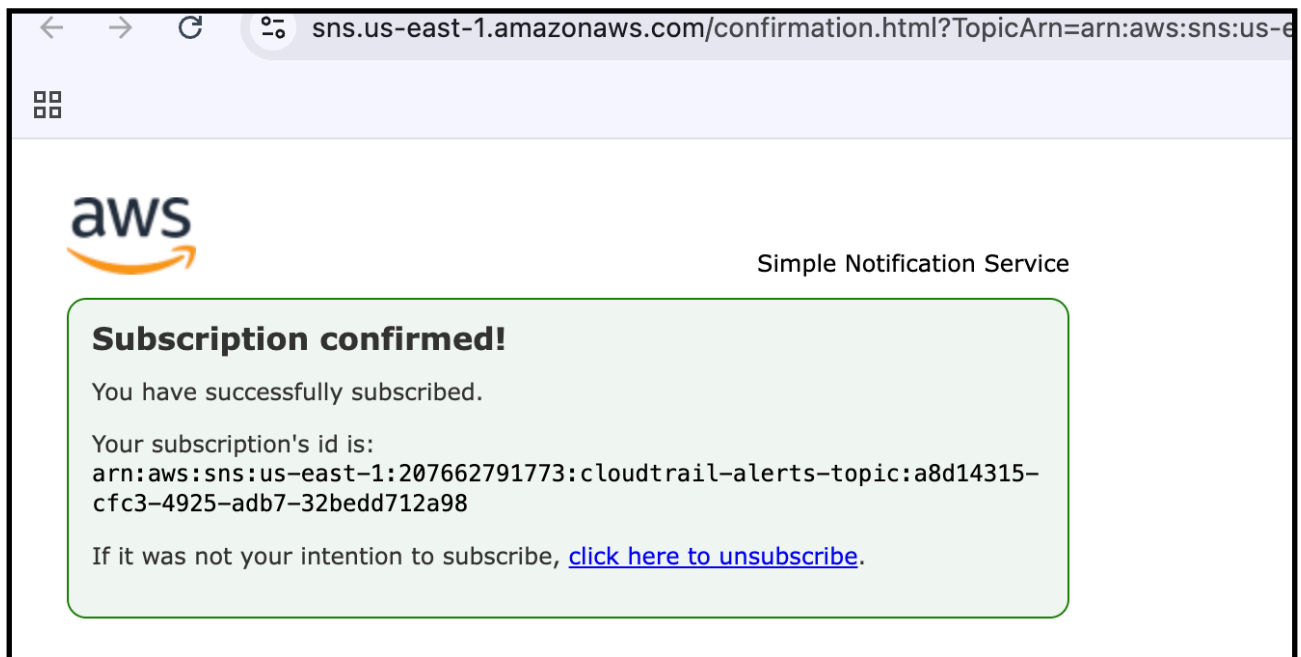
—> now give a subscription for that sns topic
—> by this we can give subscription for which sms, emails etc to get the notifications



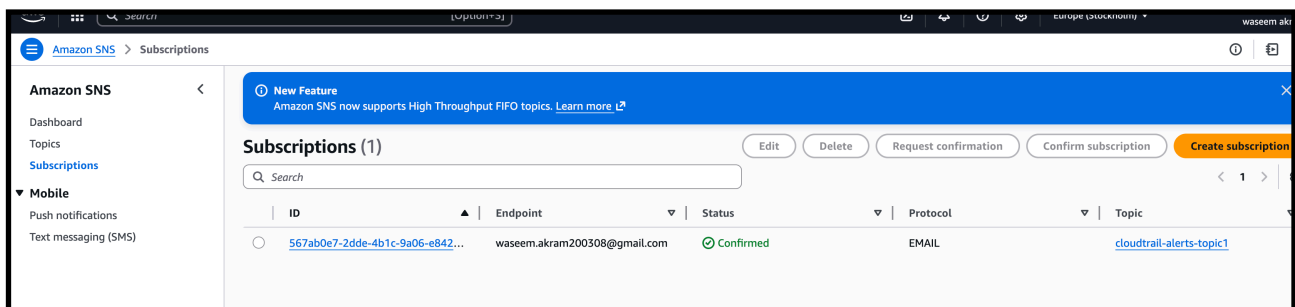
—> for now we are using email
—> and adding the details of the email name to get subscription request



—> after that check the gmail of the email we given we have to accept the subscription and check th status



—> it will show the status as confirm if we subscribe



—> now we have to create a cloud trail and attach the sns for it

—> give a s3 bucket to create it

The screenshot shows the 'Choose trail attributes' step in the AWS CloudTrail console. The left sidebar indicates the current step is 'Choose trail attributes'. The main content area is titled 'Choose trail attributes' and contains several sections:

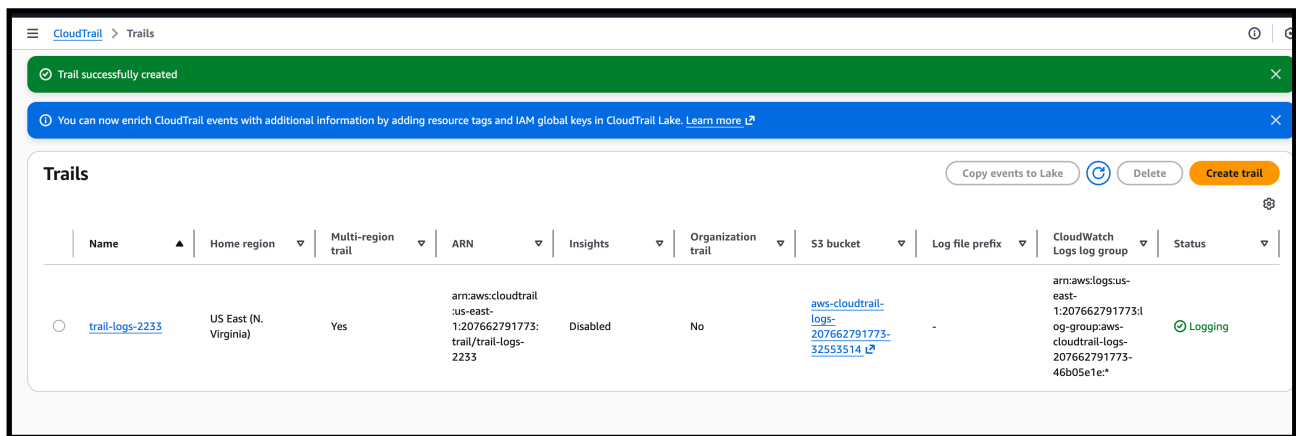
- General details:** A trail created in the console is a multi-region trail. [Learn more](#)
- Trail name:** Enter a display name for your trail. The text 'trail-logs-2233' is entered in the input field. Below the field, it states: '3-128 characters. Only letters, numbers, periods, underscores, and dashes are allowed.'
- Storage location:** [Info](#)
 - ☒ **Create new S3 bucket**
Create a bucket to store logs for the trail.
 - ☐ **Use existing S3 bucket**
Choose an existing bucket to store logs for this trail.
- Trail log bucket and folder:** Enter a new S3 bucket name and folder (prefix) to store your logs. Bucket names must be globally unique. The text 'aws-cloudtrail-logs-207662791773-32553514' is entered in the input field. Below the field, it states: 'Logs will be stored in aws-cloudtrail-logs-207662791773-32553514/AWSLogs/207662791773'.
- Log file SSE-KMS encryption:** [Info](#)
 - ☐ Enabled
- Additional settings:**
 - Log file validation:** [Info](#)

—> we can see the sns we created is available now select it

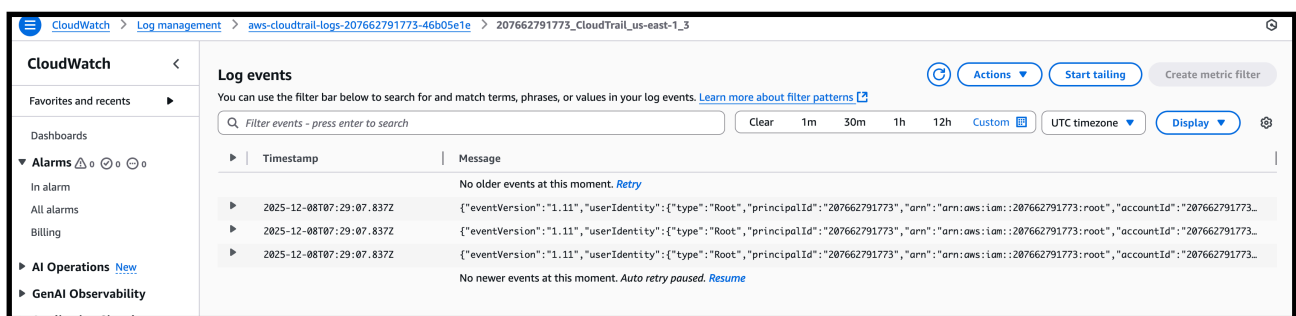
The screenshot shows the 'Additional settings' and 'CloudWatch Logs' sections of the AWS CloudTrail console. The left sidebar indicates the current step is 'Choose trail attributes'. The main content area is titled 'Additional settings' and contains several sections:

- Log file validation:** [Info](#)
 - ☒ Enabled
- SNS notification delivery:** [Info](#)
 - ☒ Enabled
- Create a new SNS topic:**
 - ☐ New
 - ☒ Existing
- SNS topic:**
- CloudWatch Logs - optional:** Configure CloudWatch Logs to monitor your trail logs and notify you when specific activity occurs. Standard CloudWatch and CloudWatch Logs charges apply. [Learn more](#)
- CloudWatch Logs:** [Info](#)
 - ☒ Enabled
- Log group:** [Info](#)
 - ☒ New
 - ☐ Existing
- Log group name:**
1-512 characters. Only letters, numbers, dashes, underscores, forward slashes, and periods are allowed.
- IAM Role:** [Info](#)
AWS CloudTrail assumes this role to send CloudTrail events to your CloudWatch Logs log group.
 - ☒ New
 - ☐ Existing
- Role name:**

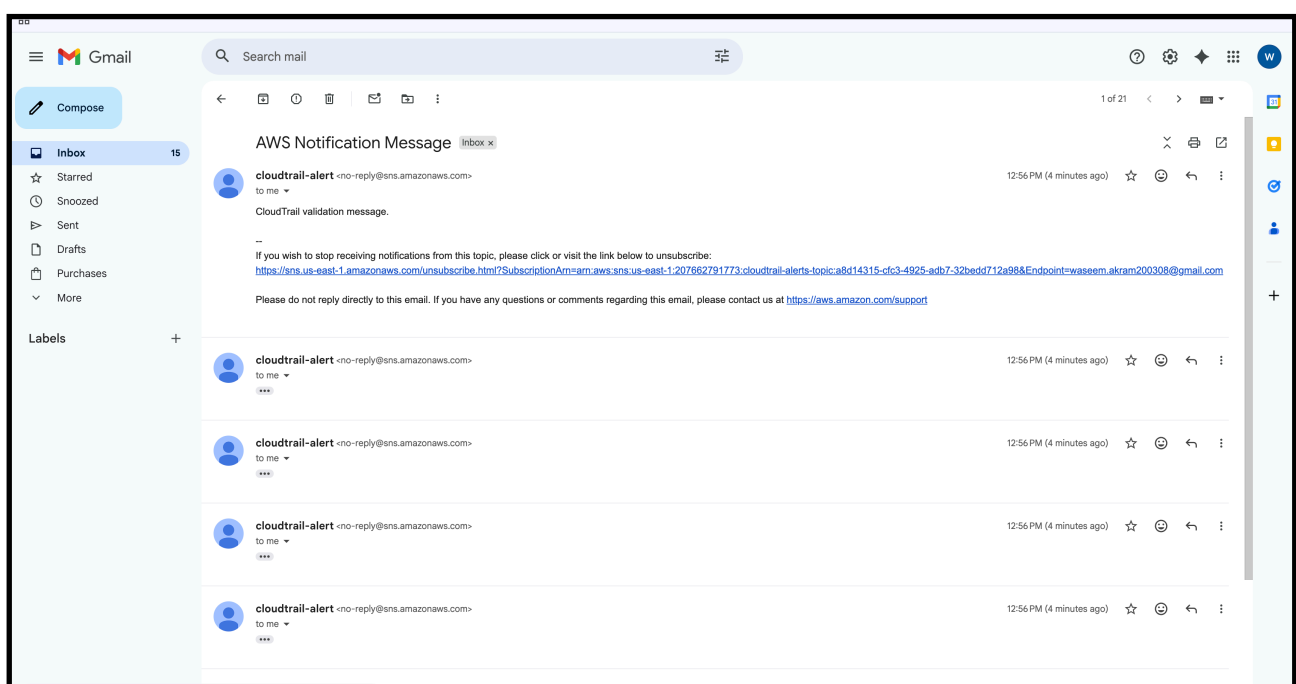
—> and created the cloutrail with sns



—> this are our cloud watch logs



—> this are cloud trail logs in aur email

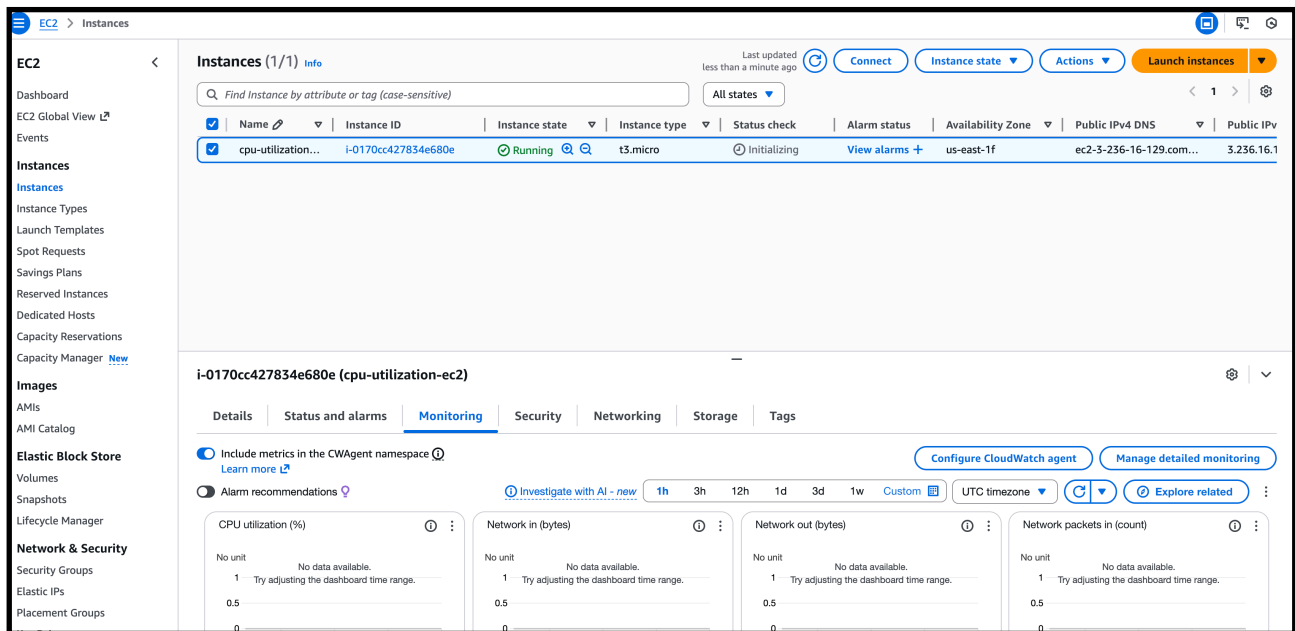


3) Configure cloud watch monitoring and record the cpu utilization and other metrics of ec2

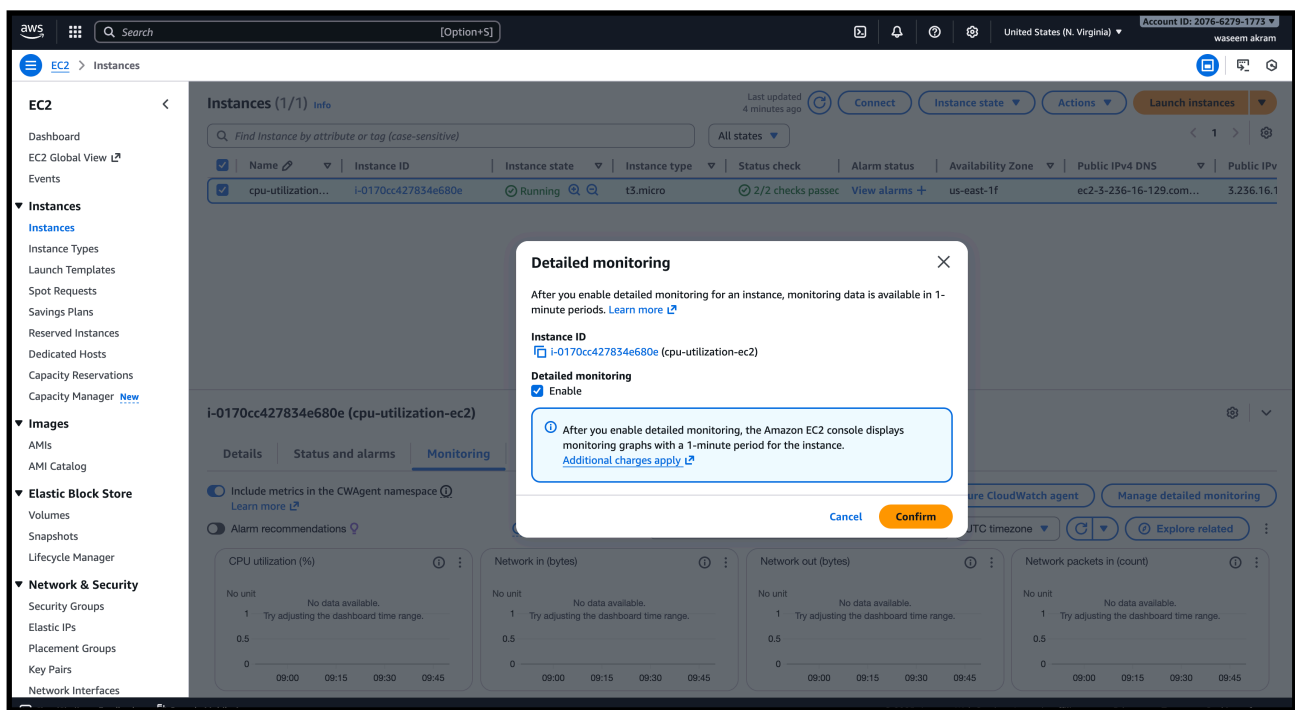
AWS already records EC2 metrics automatically.

You only need to **enable detailed monitoring** + **see the metrics**.

—> first we have to launch one instance and name it according to the task for cpu

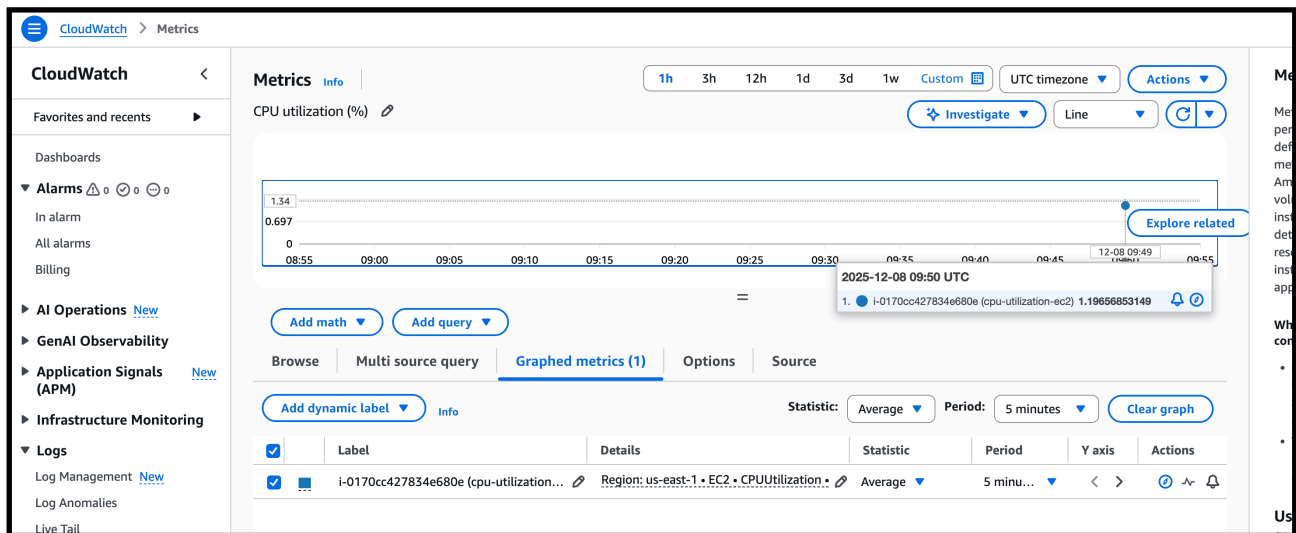


—> for monitoring the instance cpu utilization we have to click the instance—> monitoring —> and manage detailed monitoring
—> now we have to enable the monitoring for this instance



—>now we have to add some load to view the cpu utilization

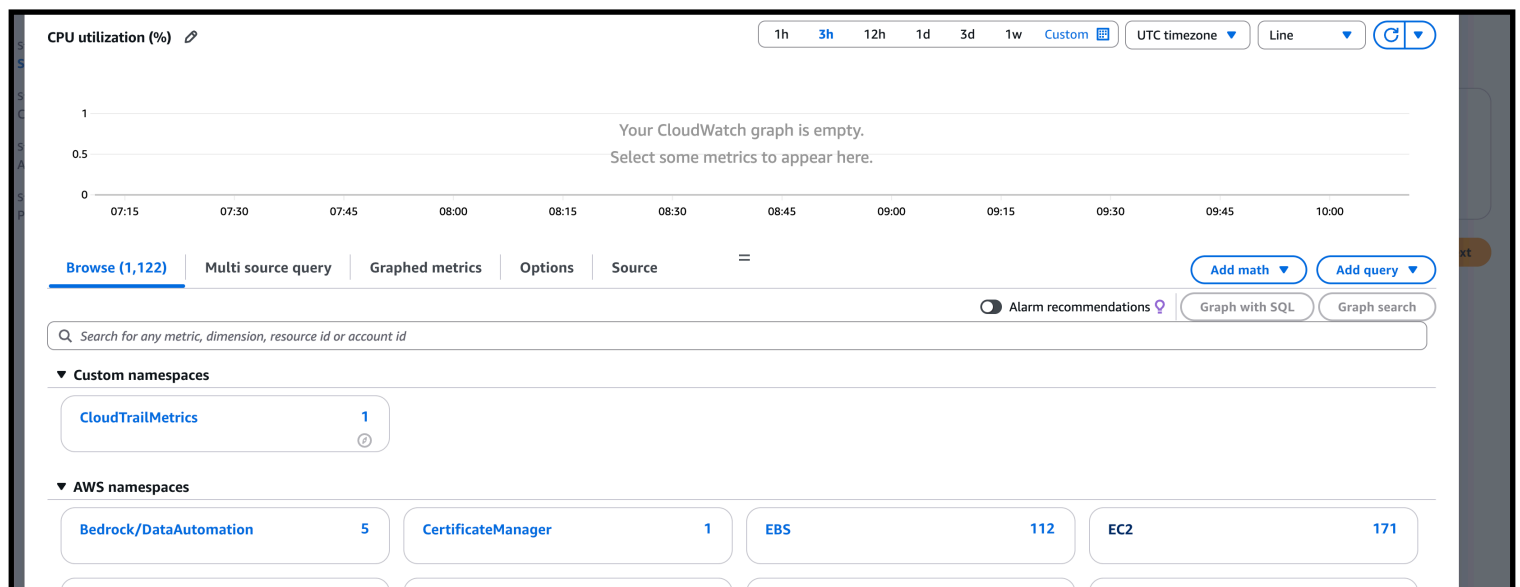
—> after adding any service inside ec2 check the monitoring of the cpu



4) Create one alarm to send alert to email if the cpu utilization is more than 70 percent

—> go to the alarm —> create alarm

—> select metric —> ec2



—> set at inalarm for getting sns notification from gmail

The screenshot shows the 'Configure actions' step in the AWS CloudWatch console. The left sidebar indicates the current step is 'Configure actions' (Step 2). The main content area is titled 'Configure actions' and contains a 'Notification' section. Under 'Alarm state trigger', the 'In alarm' option is selected, indicating the metric is outside the defined threshold. Other options are 'OK' (metric within threshold) and 'Insufficient data' (alarm just started or not enough data). Below this, the 'Send a notification to the following SNS topic' section is active, with 'Select an existing SNS topic' chosen. A search bar shows 'cloudtrail-alerts-topic' selected. The 'Email (endpoints)' section lists 'waseem.akram200308@gmail.com' with a link to 'View in SNS Console'. A 'Remove' button is visible in the top right corner of the notification section.

—> name the alarm according to task “cpu”

—> click next

The screenshot shows the 'Add alarm details' step in the AWS CloudWatch console. The left sidebar indicates the current step is 'Add alarm details' (Step 3). The main content area is titled 'Add alarm details' and contains a 'Name and description' section. The 'Alarm name' field contains 'cpu-utilization->70'. The 'Alarm description - optional' field contains a markdown-formatted text: '# This is an H1', '**double asterisks will produce strong character**', and 'This is [an example](https://example.com/) inline link.' Below the description field, a note states: 'Markdown formatting is only applied when viewing your alarm in the console. The description will remain in plain text in the alarm notifications.' A 'View formatting guidelines' link is also present.

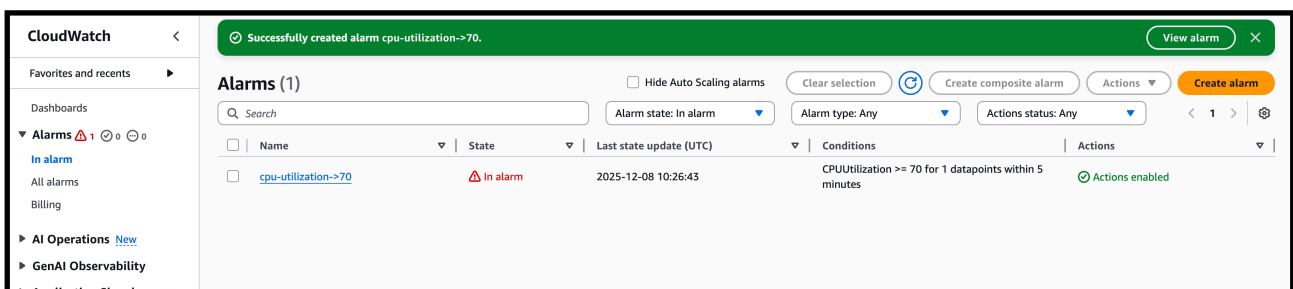
—> after clicking create . our alarm for cpu utilization is successfully created

The screenshot shows the 'Alarms' page in the AWS CloudWatch console. The left sidebar shows the 'Alarms' section with a count of 1 alarm. The main content area is titled 'Alarms (1)' and contains a table of alarms. The table has columns for 'Name', 'State', 'Last state update (UTC)', 'Conditions', and 'Action'. The first row shows the alarm 'cpu-utilization->70' with a state of 'OK' and a last state update of '2025-12-08 10:14:43'. The conditions are 'CPUUtilization >= 70 for 1 datapoints within 5 minutes'. The action is 'Act'.

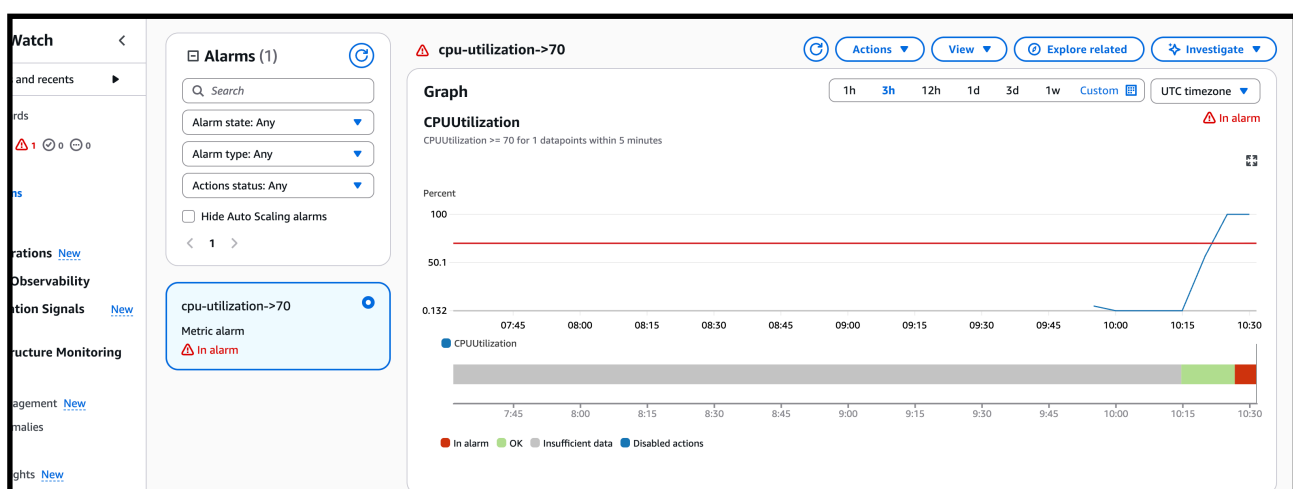
Name	State	Last state update (UTC)	Conditions	Action
cpu-utilization->70	OK	2025-12-08 10:14:43	CPUUtilization >= 70 for 1 datapoints within 5 minutes	Act

—> connect your ec2 instance and add some fake load
—> aur ec2 load is less than 70 so to add some fake load we can use
“Yes > /dev/null & “ to add some fake load to ec2

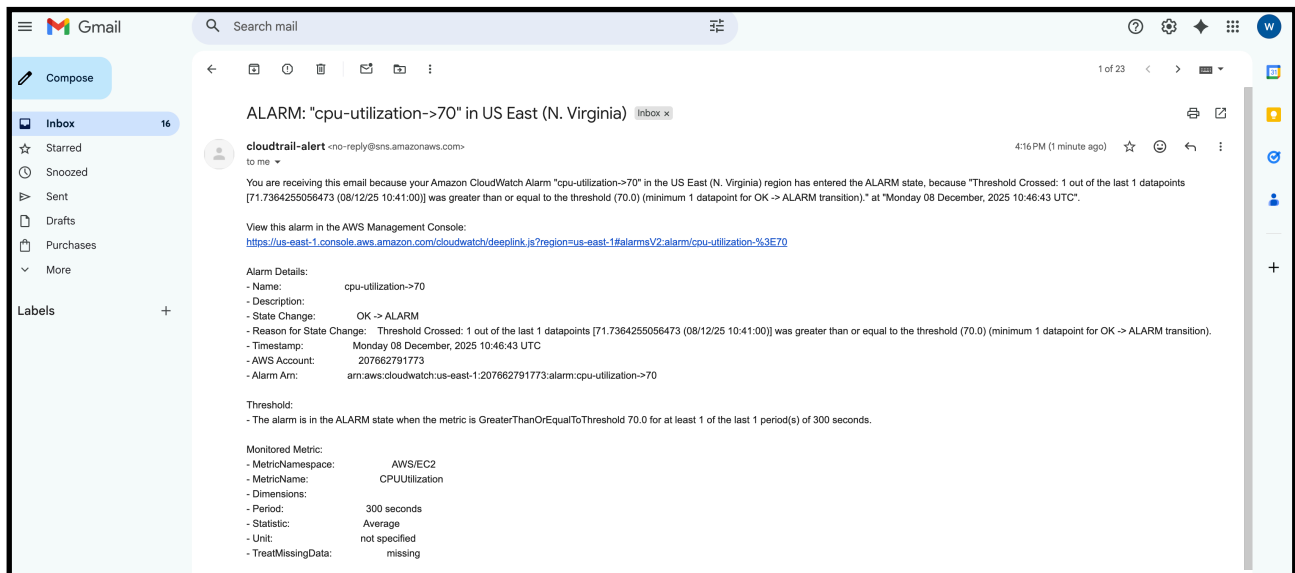
```
Complete!  
[ec2-user@ip-172-31-75-244 ~]$ yes > /dev/null &  
yes > /dev/null &  
yes > /dev/null &  
yes > /dev/null &  
[1] 26377  
[2] 26378  
[3] 26379  
[4] 26380  
[ec2-user@ip-172-31-75-244 ~]$
```



—> now check the cloud watch of the metric of cpu utilization
—> we can see the load is more than 70 now we will be getting a notification alarm in our gmail



—> I have opened the email send by aws of cpu utilization more than 70



5) Create Dashboard and monitor tomcat service wether it is running or not and send the alert.

Follow all the below steps for the task

1. Launch an EC2 instance

```
THIS KEY IS NOT KNOWN BY ANY OTHER NAMES:
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-44-210-69-76.compute-1.amazonaws.com' (ED25519) to the list of known hosts.

#_
~\ ##### Amazon Linux 2023
~~ \#####\
~~ \###|
~~ \#/ ____ https://aws.amazon.com/linux/amazon-linux-2023
~~ V~' '->
~~~
~~~.~.~
~~~/_/_/_
~~~/_m/'

Last login: Mon Dec 8 10:45:24 2025 from 18.206.107.28
[ec2-user@ip-172-31-75-244 ~]$ sudo yum update -y
Last metadata expiration check: 2:00:13 ago on Mon Dec 8 09:57:00 2025.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-75-244 ~]$ sudo -i
```

—> Install Java and Tomcat

```

[root@ip-172-31-75-244 ~]# sudo dnf install -y java-17-amazon-corretto
Last metadata expiration check: 2:03:08 ago on Mon Dec 8 09:57:00 2025.
Dependencies resolved.
=====
Package                                                                 Architecture
=====
Installing:
  java-17-amazon-corretto                                             x86_64
Installing dependencies:
  alsa-lib                                                             x86_64
  cairo                                                                 x86_64
  dejavu-sans-fonts                                                  noarch
  dejavu-sans-mono-fonts                                             noarch
  dejavu-serif-fonts                                                 noarch

```

-> sudo dnf install -y tomcat
./startup.sh to start the tomcat

```

Complete!
[[root@ip-172-31-75-244 ~]# wget https://d1cdn.apache.org/tomcat/tomcat-9/v9.0.112/bin/apache-tomcat-9.0.112.tar.gz
--2025-12-08 12:01:09-- https://d1cdn.apache.org/tomcat/tomcat-9/v9.0.112/bin/apache-tomcat-9.0.112.tar.gz
Resolving d1cdn.apache.org (d1cdn.apache.org)... 151.101.2.132, 2a04:4e42::644
Connecting to d1cdn.apache.org (d1cdn.apache.org)|151.101.2.132|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 13043951 (12M) [application/x-gzip]
Saving to: 'apache-tomcat-9.0.112.tar.gz'

apache-tomcat-9.0.112.tar.gz                                     100%[=====
2025-12-08 12:01:09 (186 MB/s) - 'apache-tomcat-9.0.112.tar.gz' saved [13043951/13043951]

[[root@ip-172-31-75-244 ~]# ls
apache-tomcat-9.0.112.tar.gz
[[root@ip-172-31-75-244 ~]# mv apache-tomcat-9.0.112.tar.gz /opt
[[root@ip-172-31-75-244 ~]# ls
[[root@ip-172-31-75-244 ~]# cd opt
-bash: cd: opt: No such file or directory
[[root@ip-172-31-75-244 ~]# cd /opt
[[root@ip-172-31-75-244 opt]# ls
apache-tomcat-9.0.112.tar.gz  aws
[[root@ip-172-31-75-244 opt]# tar xvf apache-tomcat-9.0.112.tar.gz
apache-tomcat-9.0.112/conf/
apache-tomcat-9.0.112/conf/catalina.policy

```

```

BUILDING.txt  CONTRIBUTING.md  LICENSE  NOTICE  README.md  RELEASE-NOTES  RUNNING.txt  bin  conf  lib  logs  temp  webapps  work
[[root@ip-172-31-75-244 apache-tomcat-9.0.112]# cd bin
[[root@ip-172-31-75-244 bin]# ./startup.sh
Using CATALINA_BASE:   /opt/apache-tomcat-9.0.112
Using CATALINA_HOME:   /opt/apache-tomcat-9.0.112
Using CATALINA_TMPDIR: /opt/apache-tomcat-9.0.112/temp
Using JRE_HOME:        /usr
Using CLASSPATH:       /opt/apache-tomcat-9.0.112/bin/bootstrap.jar:/opt/apache-tomcat-9.0.112/bin/tomcat-juli.jar
Using CATALINA_OPTS:
Tomcat started.
[[root@ip-172-31-75-244 bin]#

```


2) -> Create Tomcat monitoring script

```
#!/bin/bash

# Check if Tomcat is running
if pgrep -f tomcat > /dev/null
then
    STATUS=1
else
    STATUS=0
fi

# Send custom metric to CloudWatch
aws cloudwatch put-metric-data \
    --metric-name tomcat_status \
    --namespace "CustomTomcat" \
    --value $STATUS \
    --region us-east-1

~
~
~
~
~
~
~
~
~
```

```
[[root@ip-172-31-75-244 bin]# cd
[root@ip-172-31-75-244 ~]# sudo vi monitoring.bash
[[root@ip-172-31-75-244 ~]# chmod 755 monitoring.bash
```

--> Give execution permission
Chmod 755 for this script file

3) aws configure

-> inside your ec2 give aws configure to access the root user of your account

```
[root@ip-172-31-75-244 ~]# sudo vi monitoring.bash
[root@ip-172-31-75-244 ~]# chmod 755 monitoring.bash
[root@ip-172-31-75-244 ~]# aws configure
[AWS Access Key ID [None]: AKIATAWNKXROZTWVRRMS
[AWS Secret Access Key [None]: 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
[Default region name [None]: us-east-1
[Default output format [None]: json
[root@ip-172-31-75-244 ~]# ./monitoring.bash
```

-> after aws configuring we have to run the ./monitoring.bash script to. Run the script

-> this script will create a matrix for the tomcat

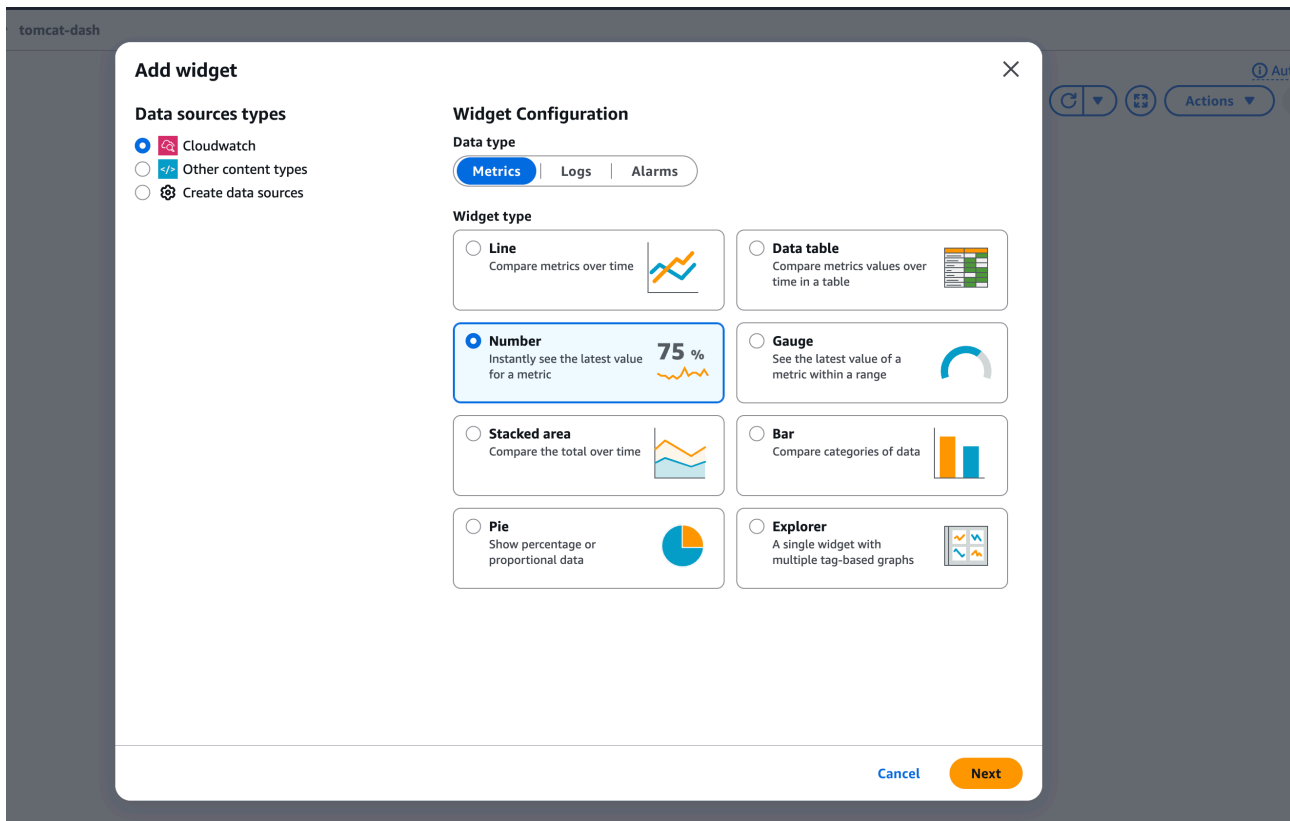
-> next to check the matrix we created by script

-> go to cloudwatch -> metrics -> browse -> custom tomcat

-> we are able to see the matrix which we wanted to monitor

The screenshot shows the AWS CloudWatch Metrics console. The left sidebar contains navigation links: Favorites and recents, Dashboards, Alarms (0), AI Operations (New), GenAI Observability, and Application Signals (APM). The main content area is titled 'Metrics' and shows 'CPU utilization (%)' as the selected metric. Below this, there are tabs for 'Browse (1,125)', 'Multi source query', 'Graphed metrics (0/1)', 'Options', and 'Source'. The 'Browse' tab is active, showing a search bar and a list of namespaces. Under 'Custom namespaces', there is a card for 'CustomTomcat' with a count of 1. Under 'AWS namespaces', there are cards for 'Bedrock/DataAutomation' (5) and 'CertificateManager' (1). An 'EBS' section is also visible with a link to 'View automatic dashboard'.

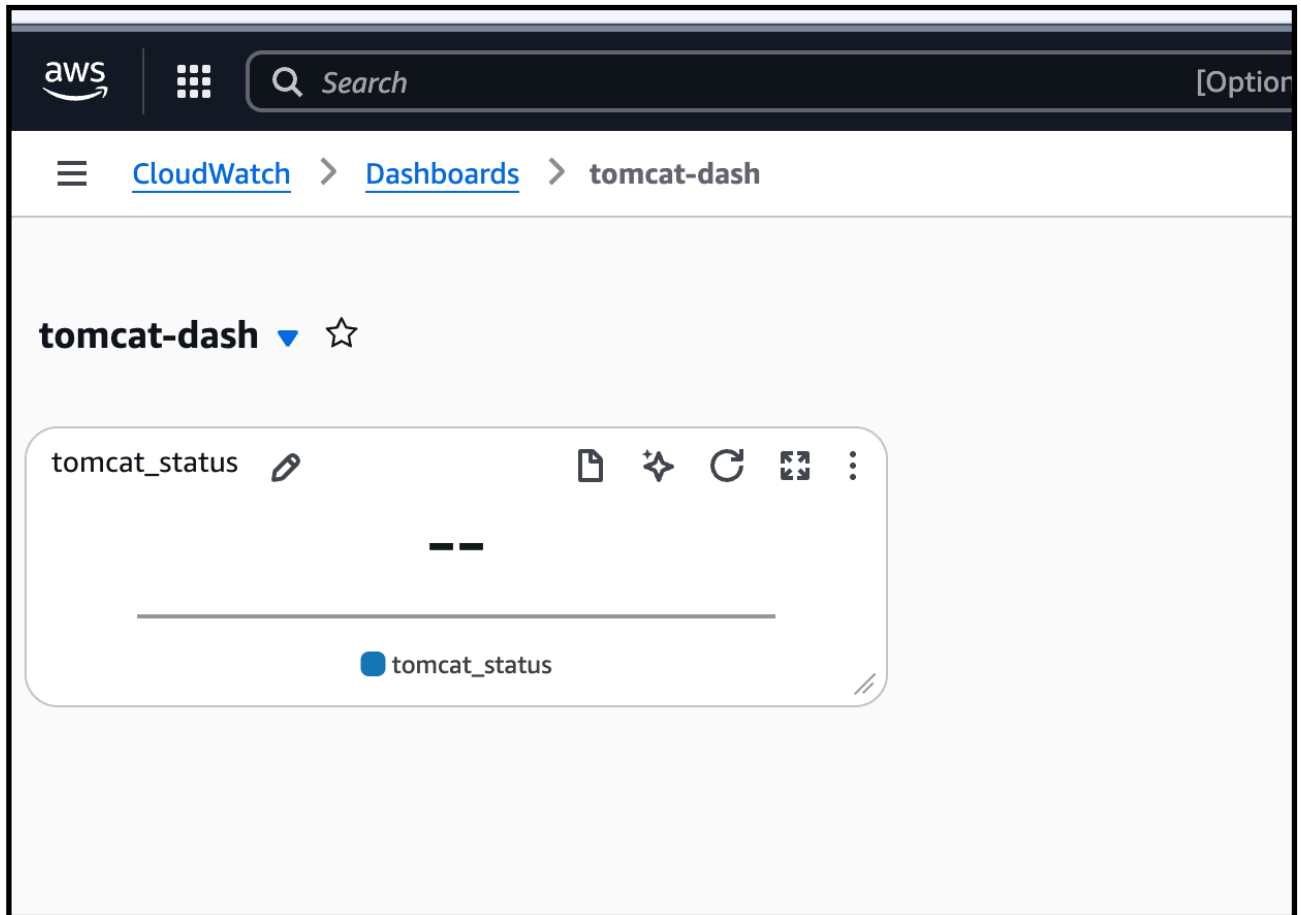
- 4)-> now create a dashboard
- > select the custom tomcat
 - > give numeric for better understanding



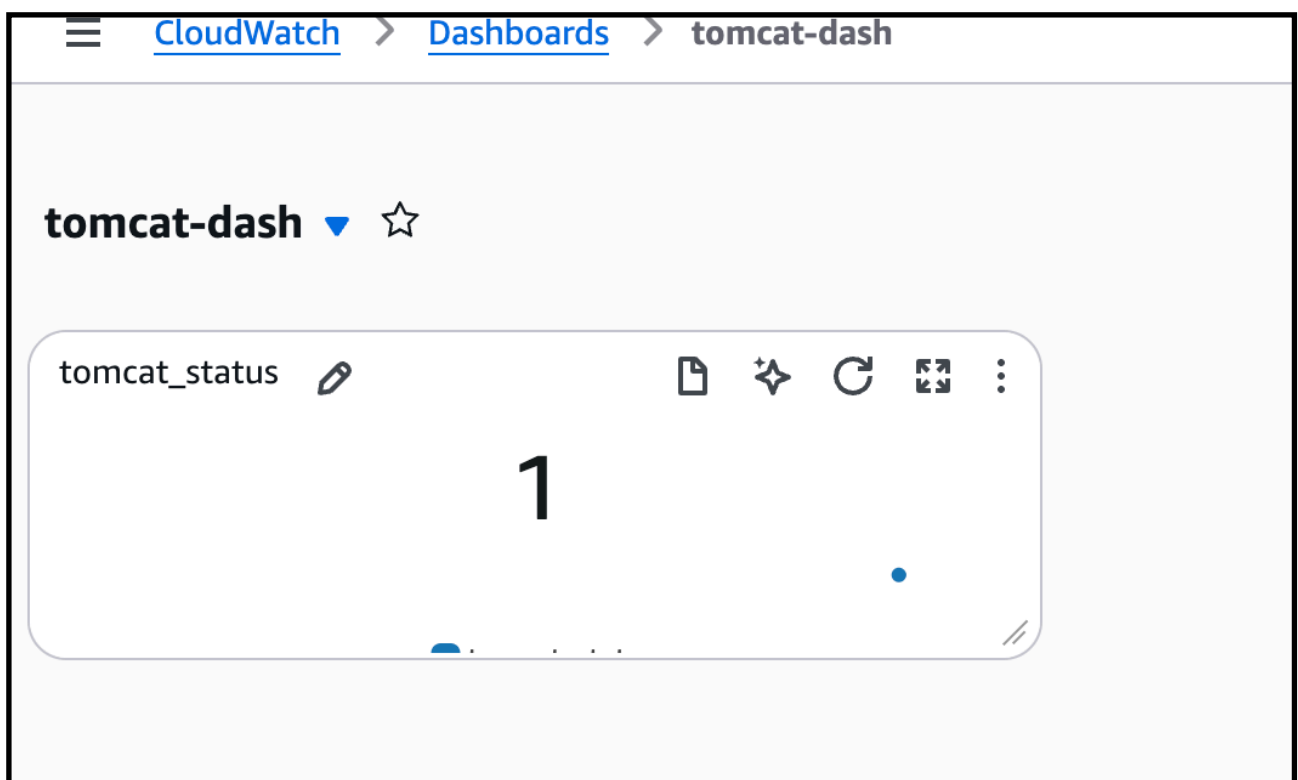
- > click next after selecting the metric

-> now we can see dashboard of the tomcat is available

-> now refresh after 1 min



-> it will show 1 because tomcat it is active



5)—>Configure cron job to send metric every minute

-> crontab -e to add the cron job

To send metric every minute

```
* * * * * /root/monitoring.bash
```

-> crontab -l for the list of cronjob

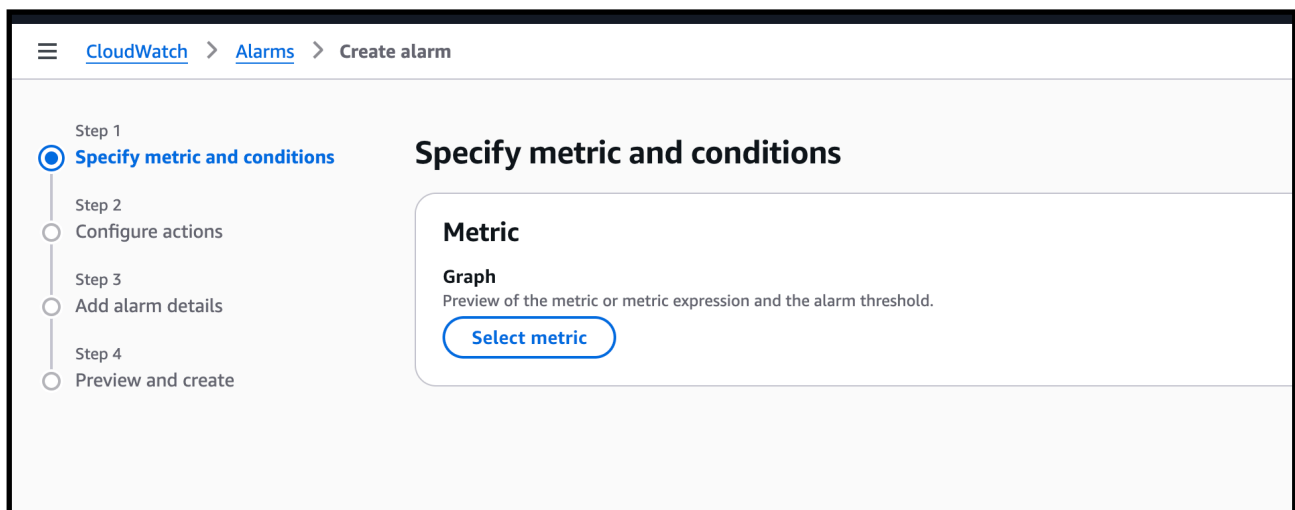
```
[root@ip-172-31-75-244 ~]# crontab -e
no crontab for root - using an empty one
crontab: installing new crontab
[[root@ip-172-31-75-244 ~]# crontab -l
* * * * * /root/monitoring.bash

[[root@ip-172-31-75-244 ~]# cd bin
-bash: cd: bin: No such file or directory
[[root@ip-172-31-75-244 ~]# ls
monitoring.bash
```

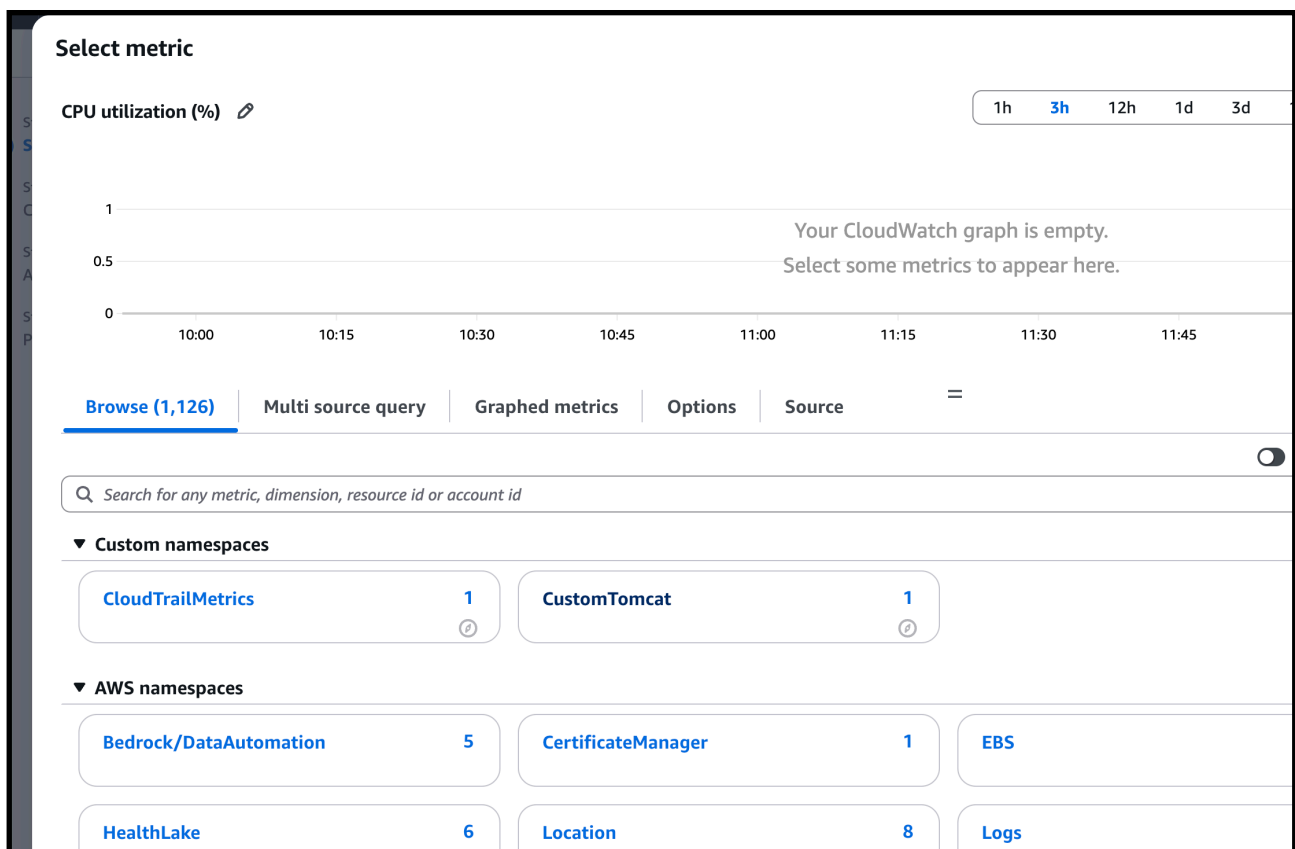
6) -> now lets create a alarm to get email when tomcat is shutdown
0 if tomcat status is zero ion dash board then send a email

-> alarm -> create alarm -> select metric

-> we have to select the tomcat metric which we created



-> select the custom tomcat



-> now give the conditions of the alarm
-> we want to get alarm notification when our dashboard of tomcat is 0 (service stopped)

No unit

1

0.9

0.8

10:00 10:30 11:00 11:30 12:00 12:30

tomcat_status

CustomTomcat

Metric name

tomcat_status

Statistic

Average

Period

5 minutes

Conditions

Threshold type

☒ Static
Use a value as a threshold

☐ Anomaly detection
Use a band as a threshold

Whenever tomcat_status is...

Define the alarm condition.

☐ Greater
> threshold

☐ Greater/Equal
>= threshold

☐ Lower/Equal
<= threshold

☒ Lower
< threshold

than...

Define the threshold value.

1

Must be a number.

-> now in configure actions step
-> alarm start trigger -> select in alarm

CloudWatch > Alarms > Create alarm

Step 1
Specify metric and conditions

Step 2
Configure actions

Step 3
Add alarm details

Step 4
Preview and create

Configure actions

Notification

Alarm state trigger

Define the alarm state that will trigger this action.

☒ In alarm
The metric or expression is outside of the defined threshold.

☐ OK
The metric or expression is within the defined threshold.

☐ Insufficient data
The alarm has just started or not enough data is available.

Remove

Send a notification to the following SNS topic

Define the SNS (Simple Notification Service) topic that will receive the notification.

☒ Select an existing SNS topic

☐ Create new topic

☐ Use topic ARN to notify other accounts

Send a notification to...

cloudtrail-alerts-topid

Only topics belonging to this account are listed here. All persons and applications subscribed to the selected topic will receive notifications.

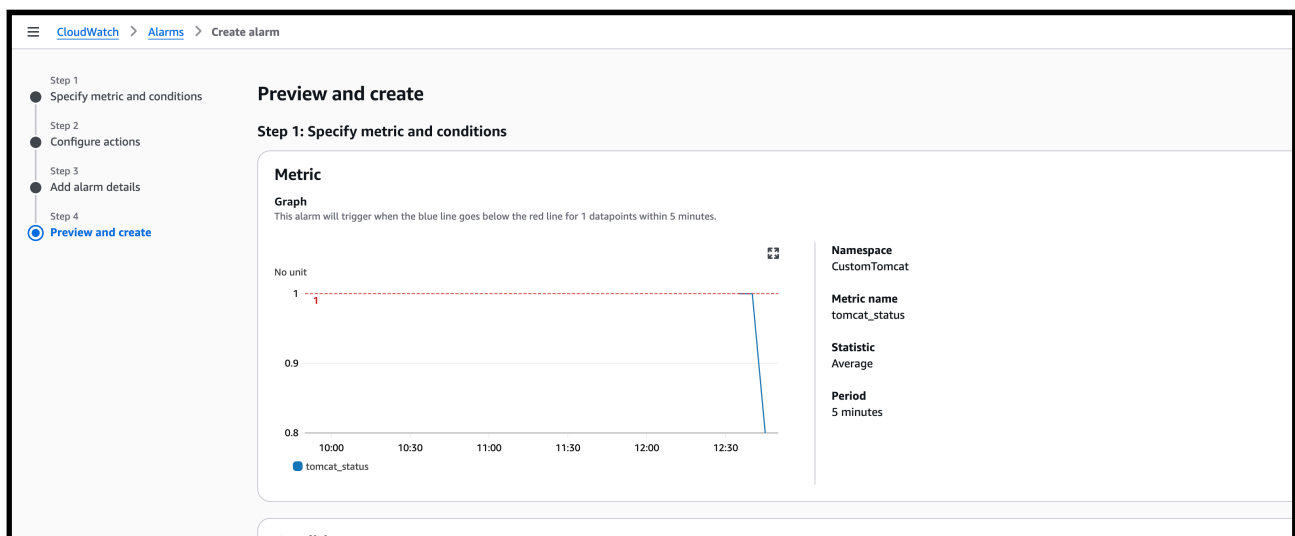
Email (endpoints)

waseem.akram200308@gmail.com - View in SNS Console

Add notification

-> select the sns for the subscription which we created to get notification on particular email
-> so by this if the tomcat is stoped meaning dashboard is 0 then we will get a trigger alarm notification on our email which we attached to sns

-> click next and preview next



-> now to check the tomcat-alarm is created now it is in alarm state because the tomcat is shutdown

Search	Alarm state: Any	Alarm type: Any	Actions status: Any	< 1 >	
Name	State	Last state update (UTC)	Conditions	Actions	
☐ nginx-alarm	OK	2025-12-08 13:07:32	nginx_status < 1 for 1 datapoints within 1 minute	Actions enabled	
☐ tomcat-alarm	In alarm	2025-12-08 12:59:11	tomcat_status < 1 for 1 datapoints within 5 minutes	Actions enabled	

-> tomcat was shutdown

```
Tomcat Shutdown:  
[root@ip-172-31-75-244 bin]# ./shutdown.sh  
Using CATALINA_BASE:   /opt/apache-tomcat-9.0.112  
Using CATALINA_HOME:   /opt/apache-tomcat-9.0.112  
Using CATALINA_TMPDIR: /opt/apache-tomcat-9.0.112/temp  
Using JRE_HOME:        /usr  
Using CLASSPATH:       /opt/apache-tomcat-9.0.112/bin/bootstrap  
Using CATALINA_OPTS:
```

-> so we will be getting a alarm notification that tomcat dashboard is at 0

tomcat-dash ▼ ☆

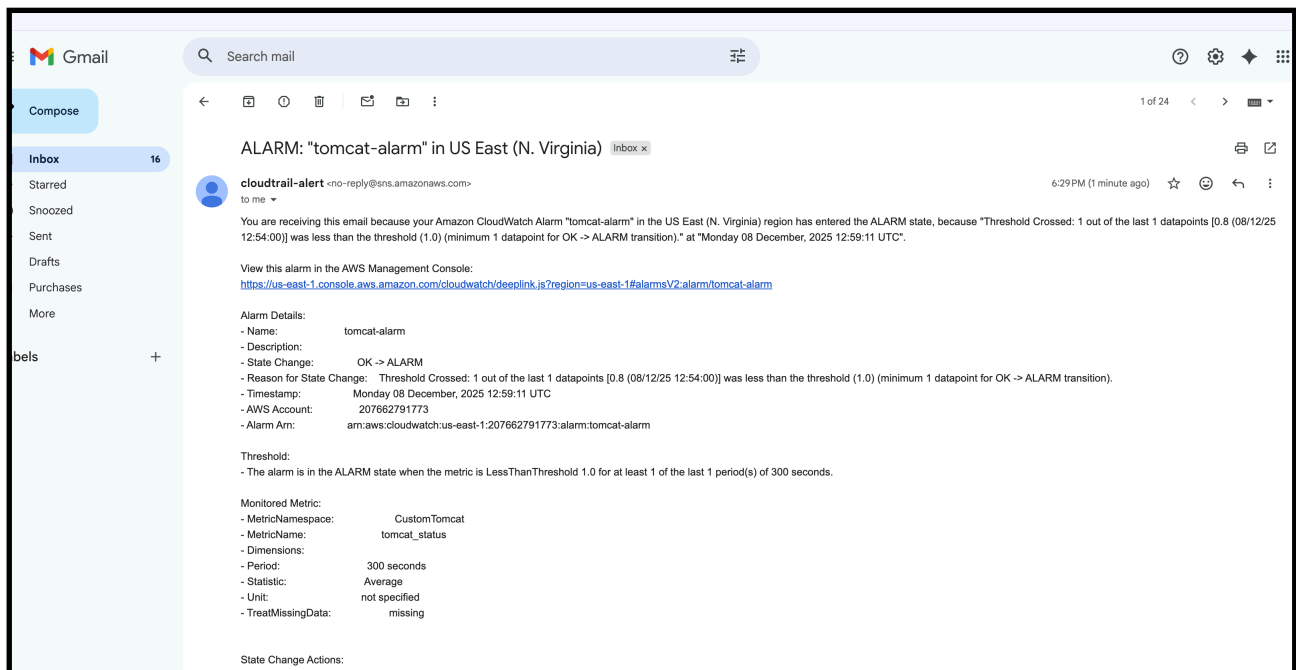
tomcat_status



0



--> and we will be getting a alarm notification on our gmail



6) Create Dashboard and monitor nginx service to send the alert if nginx is not running.

—> now same as task 5 we have to do same with nginx

—> step by step follow

—> yum install nginx -y

Systemctl start nginx

Systemctl status nginx

—> nginx is running

```
[root@ip-172-31-75-244 bin]# cd
[root@ip-172-31-75-244 ~]# sudo dnf install -y nginx
sudo systemctl start nginx
sudo systemctl enable nginx
sudo systemctl status nginx
Last metadata expiration check: 3:04:51 ago on Mon Dec  8 09:57:00 2025.
Package nginx-1:1.28.0-1.amzn2023.0.2.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service → /usr/lib/systemd/system/nginx.service
● nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
   Active: active (running) since Mon 2025-12-08 13:01:51 UTC; 356ms ago
     Main PID: 4730 (nginx)
        Tasks: 3 (limit: 1053)
       Memory: 4.9M
          CPU: 56ms
      CGroup: /system.slice/nginx.service
```

— —> after this we have too add a script for nginx to create its metric
To monitore in aur aws cloudwatch

—> create a file and add the below script to create a metric in aur cloudwatch

—> give execute permission to the file

```
#!/bin/bash

# Check if NGINX is running
if pgrep -f nginx > /dev/null
then
    STATUS=1
else
    STATUS=0
fi

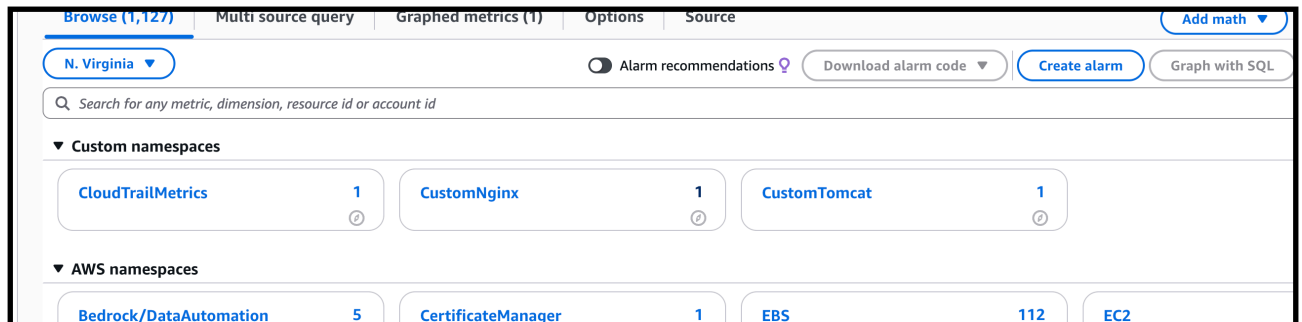
# Send custom metric to CloudWatch
aws cloudwatch put-metric-data \
    --metric-name nginx_status \
    --namespace "CustomNginx" \
    --value $STATUS \
    --region us-east-1
```

—> run the script ./monitoring_nginx.bash

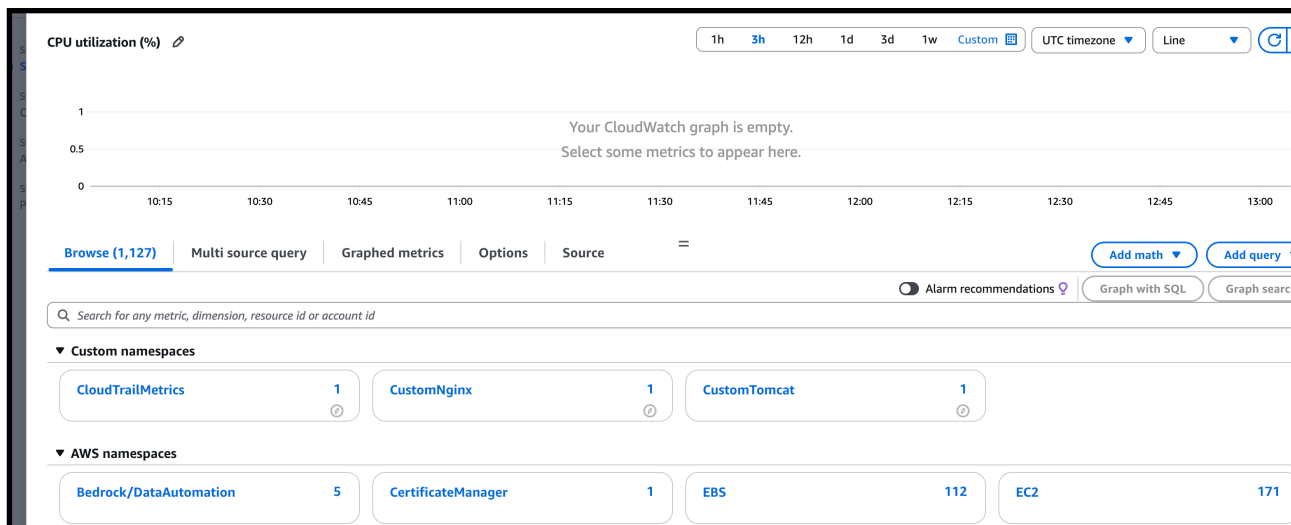
```
[root@ip-172-31-75-244 ~]# vi monitoring_nginx.bash
[root@ip-172-31-75-244 ~]# chmod +x monitoring_nginx.bash
[root@ip-172-31-75-244 ~]# ./monitoring_nginx.bash
```

--> now go and check the matrix --> browse

--> we are able to see there is customnginx matrix created by the script



--> now creates a dash board for the nginx monitoring



--> dashboard is created for nginx

Filter dashboards				
	Name	Sharing	Favorite	Last update (UTC)
☐	nginx-dash		☆	2025-12-08 13:08
☐	tomcat-dash		☆	2025-12-08 12:39

--> give the same crontab rule for nginx also to check the monitoring

```
crontab: installing new crontab
[root@ip-172-31-75-244 ~]# crontab -l
* * * * * /root/monitoring.bash
* * * * * /root/monitoring_nginx.bash

[root@ip-172-31-75-244 ~]#
```

--> now create the alarm for the nginx

CloudWatch > Alarms > Create alarm

Step 4: Review and create

No unit

2

1

0

10:30 11:00 11:30 12:00 12:30 13:00

● nginx_status

Namespace
CustomNginx

Metric name
nginx_status

Statistic
Average

Period
1 minute

Conditions

Threshold type

☒ Static
Use a value as a threshold

☐ Anomaly detection
Use a band as a threshold

Whenever nginx_status is...

Define the alarm condition.

☐ Greater
> threshold

☐ Greater/Equal
≥ threshold

☐ Lower/Equal
≤ threshold

☒ Lower
< threshold

than...

Define the threshold value.

1

Must be a number.

—> give same rules for nginx also if dashboard of the nginx is 0 trigger the alarm

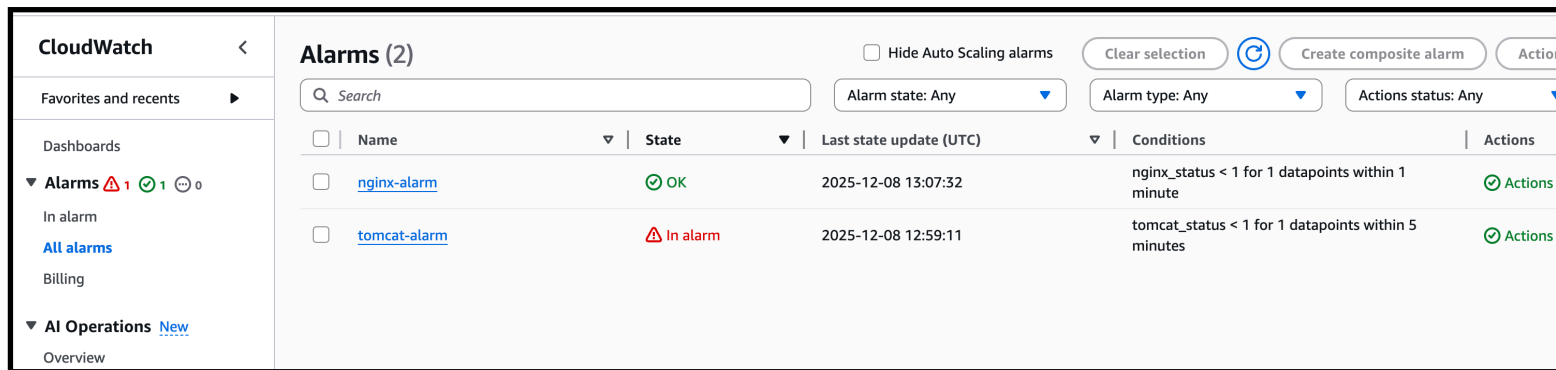
The screenshot shows the 'Configure actions' step in the AWS CloudWatch console. On the left, a progress bar indicates four steps: 'Specify metric and conditions', 'Configure actions' (current), 'Add alarm details', and 'Preview and create'. The main content area is titled 'Configure actions' and contains a 'Notification' section. Under 'Alarm state trigger', there are three radio buttons: 'In alarm' (selected), 'OK', and 'Insufficient data'. The 'In alarm' option has a description: 'The metric or expression is outside of the defined threshold.' Below this, there is a section 'Send a notification to the following SNS topic' with a description: 'Define the SNS (Simple Notification Service) topic that will receive the notification.' There are three radio buttons: 'Select an existing SNS topic' (selected), 'Create new topic', and 'Use topic ARN to notify other accounts'. Below these is a search bar with the text 'cloudtrail-alerts-topic' and a close button. A note below the search bar states: 'Only topics belonging to this account are listed here. All persons and applications subscribed to the selected topic will receive notifications.' At the bottom, there is an 'Email (endpoints)' section showing 'waseem.akram200308@gmail.com - View in SNS Console' and an 'Add notification' button.

—> select the sns for email notification process

The screenshot shows the 'Add alarm details' step in the AWS CloudWatch console. On the left, a progress bar indicates four steps: 'Specify metric and conditions', 'Configure actions', 'Add alarm details' (current), and 'Preview and create'. The main content area is titled 'Add alarm details' and contains a 'Name and description' section. Under 'Alarm name', there is a text input field with the value 'nginx-alarm'. Below this is a section 'Alarm description - optional' with a link 'View formatting guidelines'. There are two tabs: 'Edit' (selected) and 'Preview'. The 'Edit' tab shows a text area with the following content: '# This is an H1', '**double asterisks will produce strong character**', and 'This is [an example](https://example.com/) inline link.'.

—> give the alarm name to nginx

—-> after alarm create we have set the alarm

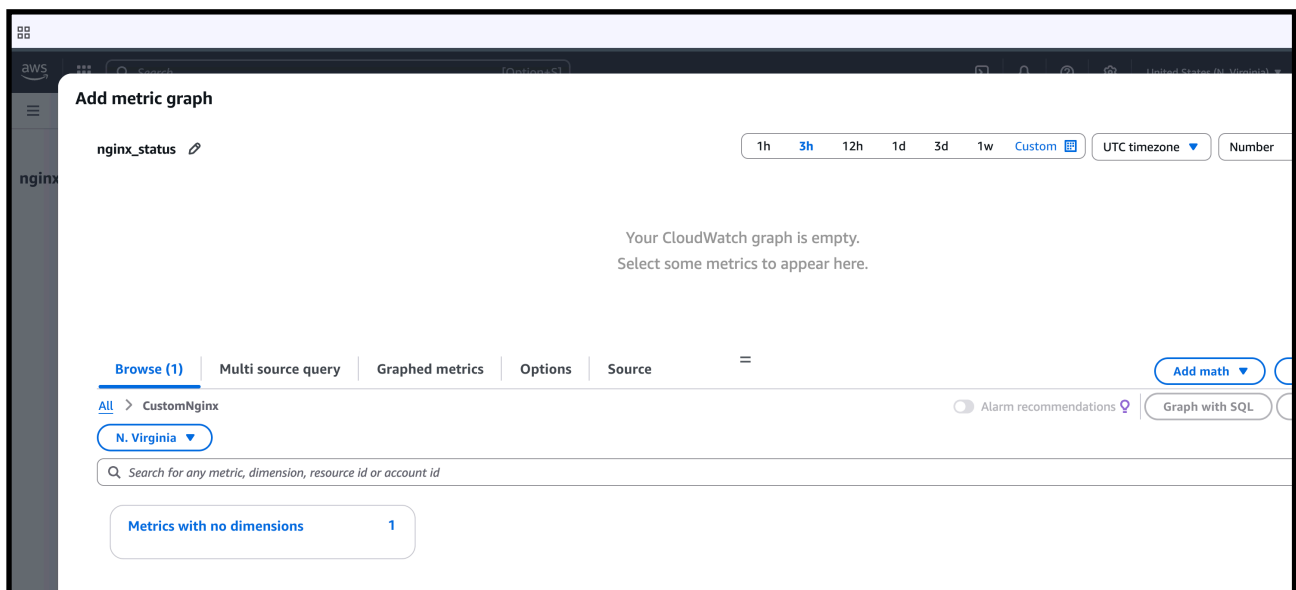


The screenshot shows the AWS CloudWatch Alarms console. On the left, there's a sidebar with 'CloudWatch' and 'Alarms (2)'. The main area displays a table of alarms. The 'nginx-alarm' is in an 'OK' state, and the 'tomcat-alarm' is in an 'In alarm' state.

☐	Name	State	Last state update (UTC)	Conditions	Actions
☐	nginx-alarm	OK	2025-12-08 13:07:32	nginx_status < 1 for 1 datapoints within 1 minute	Actions
☐	tomcat-alarm	In alarm	2025-12-08 12:59:11	tomcat_status < 1 for 1 datapoints within 5 minutes	Actions

—->to check our nginx dashboard

—> selected the dashboard



—-> and select the metric it will show the dash board value as 1 because the nginx is running

- > nginx is running if the nginx is stoped the status will be 0
- > and we will get the alarm notification to the email same as tomcat from task 4

